📖 DAY 1 MASTER TERMINOLOGY & CONCEPT GLOSSARY

Complete HPC, Linux, Shell Scripting, SSH & Intel oneAPI Dictionary

Course Code: `GLA-HPC-BCSG701` | **Track:** Intel & NEC Supercomputing Alliance
Lead Trainer: Shivam Gupta | **Target Session:** Day 1 (Full 6 Hours: 3h Morning Architecture + 3h Afternoon Hands-On Lab)
Primary Reference: `Intel & NEC - High Performance Computing.pdf` (Page 2)

📌 How to Use This Master Glossary

This document is your **single-source comprehensive encyclopedia** for Day 1. Every single technical term, hardware architecture acronym, Linux command flag, networking concept, and Intel compiler directive that you will utter during your 6-hour masterclass is defined here.

Each entry includes:

1. **Full Form & Core Definition**
2. **Layman Analogy (Hindi + English)** for intuitive classroom delivery
3. **Underlying Technical Mechanism (Hardware / Silicon / OS level)**
4. **Classroom Context & Trainer Script** (What to say when students ask!)

📄 TABLE OF CONTENTS BY SYLLABUS TOPIC

- Module 1: Introduction to HPC & Supercomputing Foundations
- Module 2: Intel HPC Hardware & Silicon Architecture
- Module 3: Linux Operating System & File System Internals
- Module 4: Shell Scripting, Streams & Automation
- Module 5: Secure Shell (SSH) & Cluster Networking
- Module 6: Intel oneAPI Toolkit (Compilers & SYCL Programming)
- Module 7: Intel HPC Toolkit (MPI, Fortran & Profiling)

Module 1: Introduction to HPC & Supercomputing Foundations

1. HPC (High Performance Computing)

- **Full Form:** High Performance Computing
- **Layman Analogy:** Normal PC ek family car hai jo 4-5 logon ko le ja sakti hai; HPC cluster ek massive freight train ya supersonic cargo fleet hai jo hazaron tons cargo (data) ek saath move karti hai.

- **Technical Mechanism:** Aggregation of thousands of processing cores, terabytes of high-bandwidth memory (DDR5/HBM), ultra-low-latency interconnect networks, and parallel file systems operating concurrently to execute quadrillions of floating-point operations per second (10^{15} FLOPS).
- **Classroom Cue:** "*Class, jab single workstation par computational time days ya weeks ban jaaye, tab hum HPC par switch karte hain.*"

2. Supercomputer vs Cluster

- **Cluster:** A collection of independent, off-the-shelf server nodes connected via local networking that function as a single unified system.
- **Supercomputer:** Historically a tightly-coupled custom monolithic architecture (e.g., Cray); today, top supercomputers (like Aurora at Argonne National Lab) are massive heterogeneous clusters consisting of tens of thousands of Intel Xeon CPUs and Data Center GPUs.
- **Classroom Cue:** "*Har supercomputer ek cluster hai, lekin har cluster supercomputer nahi hota!*"

3. Compute Node vs Login / Head Node

- **Login / Head Node:** The public-facing gateway server where users authenticate via SSH, write code, edit scripts, compile programs, and submit jobs. **Golden Rule: Never run heavy simulations on the Login Node!**
- **Compute Node (Worker Node):** The backend workhorses where actual computational jobs, matrix multiplications, and AI training run in isolation. Compute nodes do not have public internet IP addresses; they communicate over private high-speed interconnects.
- **Classroom Cue:** "*Head node airport ka check-in counter hai; Compute nodes actual aeroplane hain jisme passengers travel karte hain.*"

4. FLOPS (Floating-Point Operations Per Second)

- **Definition:** The standard metric of mathematical computational speed in science and engineering.
- **Scales:**
  - **MegaFLOPS (MFLOPS):** 10^6 ops/sec
  - **GigaFLOPS (GFLOPS):** 10^9 ops/sec (Typical laptop CPU)
  - **TeraFLOPS (TFLOPS):** 10^{12} ops/sec (Modern workstation GPU)
  - **PetaFLOPS (PFLOPS):** 10^{15} ops/sec (Enterprise HPC Clusters)
  - **ExaFLOPS (EFLOPS):** 10^{18} ops/sec (Intel-powered Aurora Supercomputer: >1 ExaFLOP)

5. Memory Bandwidth vs Latency

- **Latency:** The physical time taken for a single data packet to travel from memory to the CPU registers (measured in nanoseconds).
- **Bandwidth:** The volume of data transferred per unit time (measured in Gigabytes per second: GB/s).
- **Classroom Analogy:** Latency is how fast water flows through a pipe (water speed); Bandwidth is the width of the pipe (pipe diameter). A laptop with narrow dual-channel DDR4 chokes on large tensors, whereas an Intel Xeon Sapphire Rapids server features 8-channel DDR5 delivering over 300 GB/s.

6. Parallel File System (Lustre / DAOS / GPFS)

- **Definition:** A high-throughput storage architecture where large files are stripped across dozens of storage controllers and NVMe disk pools so hundreds of nodes can read and write data simultaneously without creating an I/O bottleneck.
- **DAOS (Distributed Asynchronous Object Storage):** Intel's next-generation open-source software-defined object store designed specifically for Storage-Class Memory (SCM) and NVMe, providing microsecond I/O.

- **Classroom Cue:** *"Agar hard drive slow ho, toh world ka fastest CPU bhi idle baitha rahega data ka intezaar karte hue (I/O Bound)."*

Module 2: Intel HPC Hardware & Silicon Architecture

7. Intel Xeon Scalable Processors (4th Gen Sapphire Rapids / 5th Gen Emerald Rapids)

- **Architecture:** Intel's enterprise server CPU architecture built on Intel 7 process technology.
- **Core Features:** Up to 64 Golden Cove cores per socket, 8-channel DDR5 memory, PCIe Gen 5.0, CXL 1.1 interconnects, and on-die hardware acceleration engines.
- **Key Advantage:** Extremely high single-thread performance combined with deep multi-core parallel density and built-in matrix acceleration.

8. Physical Cores vs Logical Threads (SMT / Hyper-Threading)

- **Physical Core:** An independent physical hardware execution unit on the silicon die containing its own ALUs (Arithmetic Logic Units), vector registers, and L1/L2 caches.
- **Logical CPU (Hyper-Threading / SMT):** Intel Hyper-Threading Technology exposes two architectural execution contexts per physical core to hide memory pipeline latency.
- **HPC Rule of Thumb:** In compute-bound numerical HPC workloads, Hyper-Threading is often disabled in BIOS (`HT=OFF`) to eliminate cache contention between threads!

9. CPU Cache Hierarchy (L1, L2, L3 LLC)

- **L1 Cache (Split L1i / L1d):** ~32KB to 48KB per core. Latency ~4 clock cycles (~1 ns). Fastest memory on Earth.
- **L2 Cache:** 2MB per core on Sapphire Rapids. Latency ~14 clock cycles (~3.5 ns).
- **L3 Cache (LLC - Last Level Cache):** Shared across all cores on the die (typically 60MB to 105MB). Latency ~40-50 cycles (~12 ns).
- **Cache Line:** Standard 64 bytes on x86_64 architecture. When the CPU requests 1 byte, the hardware prefetcher loads the entire 64-byte line into L1 cache.

10. SIMD (Single Instruction, Multiple Data) & AVX-512

- **Concept:** Instead of calculating `C[0] = A[0] + B[0]`, then `C[1] = A[1] + B[1]` in serial (SISD), SIMD calculates 16 pairs of 32-bit floats simultaneously in a single CPU cycle using 512-bit vector registers (`zmm0` - `zmm31`).
- **Intel AVX-512 (Advanced Vector Extensions 512):** Intel's premier 512-bit vector instruction set architecture.
- **VNNI (Vector Neural Network Instructions):** AVX-512 extensions designed to accelerate deep learning INT8 inference by performing 4-way fused multiply-adds.

11. Intel AMX (Advanced Matrix Extensions)

- **Definition:** Dedicated 2D hardware matrix accelerator tiles integrated directly into every physical core of 4th Gen Intel Xeon Scalable processors (Sapphire Rapids and newer).
- **Hardware Mechanism:** Uses two components:
  1. **Tiles:** 8 two-dimensional registers (1KB each = 8KB total state per core).
  2. **TMUL (Tile Matrix Multiply Unit):** A hardware accelerator capable of computing up to 2048 INT8 or 1024 Bfloat16 operations per cycle!

- **Classroom Cue:** *"AMX CPU ke andar chhota GPU bitha deta hai. AI inference CPU par hi ultra-fast ho jaata hai bina GPU khariде!"*

12. Intel Data Center GPU Max Series (Ponte Vecchio)

- **Architecture:** Intel's flagship HPC GPU built using Foveros 3D stacking and EMIB packaging, containing over 100 billion transistors across 47 active silicon tiles.
- **Core Unit (Xe-Core):** Contains 16 Vector Engines (512-bit SIMD) and 16 Matrix Engines (XMX - Xe Matrix Extensions) capable of massive FP64, FP32, and BF16 tensor math.
- **High-Bandwidth Memory (HBM2e):** Up to 128GB of stacked on-package memory providing over 3.2 Terabytes per second (TB/s) bandwidth.

13. AI Accelerators — Habana Gaudi 2 & Gaudi 3

- **Definition:** Dedicated Application-Specific Integrated Circuits (ASICs) engineered specifically for accelerating Deep Learning transformer training and inference.
- **Architecture:** Contains Tensor Processor Cores (TPC), dedicated Matrix Math Engines (MME), and HBM memory.
- **RoCE Integration:** Gaudi 2 integrates 24 ports of 100-Gigabit Ethernet (RoCEv2) directly on the processor package, eliminating expensive external network interface cards (NICs).

14. Interconnects (InfiniBand HDR/NDR vs Intel Omni-Path)

- **InfiniBand:** A high-speed, low-latency, switched-fabric communications standard.
  - **HDR:** 200 Gbps per port, <1 microsecond latency.
  - **NDR:** 400 Gbps per port.
  - **RDMA (Remote Direct Memory Access):** Allows one compute node to read/write directly into the memory of another compute node bypassing both operating system kernels!

Module 3: Linux Operating System & File System Internals

15. The Root Directory (`/`) & Linux Hierarchy

- `/` **(Root):** The apex of the single unified Linux directory tree. There are no drive letters like `C:\` or `D:\`.
- **Hierarchical Paths:**
  - `/home/username` **:** User personal space for dotfiles (`.bashrc` , `.ssh`), source code, and configuration.
  - `/etc` **:** System-wide configuration files (e.g., `/etc/hosts` , `/etc/ssh/sshd_config`).
  - `/var` **:** Variable data like logs (`/var/log`).
  - `/tmp` **:** System temporary directory (frequently wiped on reboot).
  - `/proc` **&** `/sys` **:** Virtual pseudo-filesystems exposing live kernel metrics, CPU registers, and hardware state directly as readable text.

16. HPC Cluster Specific Directories (`/home` vs `/work` vs `/scratch`)

- `/home` **:** Small quota (e.g., 20GB-50GB), backed up nightly, NFS mounted, high latency. Good for scripts. Bad for datasets!
- `/work` **:** Medium-to-large shared project directory for research teams, parallel filesystem mounted, moderately high throughput.

- `/scratch`: Massive, ultra-fast parallel storage pool (Lustre / NVMe RAID). Zero backup, purge policy (e.g., files older than 30 days are automatically deleted). **This is where high-speed simulation checkpoints and training datasets must live!**

17. POSIX (Portable Operating System Interface)

- **Definition:** An IEEE standard family defining application programming interfaces (APIs) and shell utilities so software can compile and run predictably across Unix/Linux distributions.
- **Classroom Cue:** *"POSIX standard ensure karta hai ki tumhara code Ubuntu, RHEL ya CentOS par bina change kiye chale."*

18. File Permissions & Octal Notation (rwx / 755 / 644 / 600)

Every file and directory in Linux has permissions divided into three triads: **User (Owner)**, **Group**, and **Others**.

- `r` **(Read):** Value = 4. Allows reading file content or listing directory entries.
- `w` **(Write):** Value = 2. Allows modifying file content or creating/deleting directory entries.
- `x` **(Execute):** Value = 1. Allows executing a script/binary, or entering (`cd`) a directory.
- **Common HPC Permissions:**
  - `chmod 755 script.sh` -> `rwxr-xr-x` (Owner can read/write/run; Everyone else can read/run).
  - `chmod 644 config.txt` -> `rw-r--r--` (Owner can read/write; Everyone else can read).
  - `chmod 600 id_ed25519` -> `rw-------` (Owner can read/write; Nobody else can touch! Mandatory for SSH Private Keys).
  - `chmod u+x script.sh` -> Adds execution permission specifically for the owner.

19. Essential Linux Commands Dictionary

Command	Full Meaning	Exact Syntax	Real HPC Usage
`pwd`	Print Working Directory	`pwd`	Verifies the active path before executing file modifications.
`ls`	List Directory Contents	`ls -lah`	`-l` (long listing), `-a` (show hidden `.files`), `-h` (human-readable sizes: KB/MB/GB).
`cd`	Change Directory	`cd ~/hpc_day01`	Navigates the directory tree. `cd ..` goes up one level; `cd ~` goes home.
`mkdir`	Make Directory	`mkdir -p a/b/c`	`-p` flag creates all missing parent directories without throwing errors.
`touch`	Update Timestamp / Create File	`touch run.log`	Creates an empty file if it doesn't exist, or updates its timestamp.
`cat`	Concatenate & Display	`cat file.txt`	Dumps the entire contents of a file to the terminal screen.
`head` / `tail`	Output First / Last Lines	`tail -n 20 app.log`	Inspects the newest lines of simulation log files.
`grep`	Global Regular Expression Print	`grep -E "avx512	amx" /proc/cpuinfo`
`find`	Search Filesystem Objects	`find . -name "*.cpp"`	Recursively searches the directory tree for files matching criteria.
`cp`	Copy File / Directory	`cp -r src/ dest/`	`-r` flag recursively copies directory trees.

Command	Full Meaning	Exact Syntax	Real HPC Usage
`mv`	Move / Rename	`mv old.txt new.txt`	Changes file path or renames files without copying data blocks.
`rm`	Remove	`rm -rf tmp_dir`	`-r` recursive, `-f` force. **Warning: Linux has no Recycle Bin! Deletion is permanent.**

Module 4: Shell Scripting, Streams & Automation

20. Shell vs Shell Script

- **Shell:** The interactive command-language interpreter (e.g., `bash`, `zsh`, `sh`) that reads commands from keyboard input or scripts and translates them into system calls for the Linux kernel.
- **Shell Script:** A plain-text executable file containing an ordered sequence of shell commands, variables, loops, and control logic executed sequentially.

21. Shebang (`#!/usr/bin/env bash`)

- **Definition:** The very first two bytes of an executable text script (`#!` - ASCII `0x23 0x21`).
- **Mechanism:** The Linux kernel's `execve()` system call reads the shebang line to identify which interpreter binary must be spawned to execute the script.
- **Why `#!/usr/bin/env bash` is Superior:** Hardcoding `#!/bin/bash` breaks on systems where Bash lives in `/usr/local/bin` (e.g., BSD or custom enterprise HPC environments). Using `/usr/bin/env bash` dynamically searches the user's `$PATH` for the correct Bash interpreter!

22. Standard I/O Streams (File Descriptors 0, 1, 2)

In Linux, every spawned process has three open file streams:

1. `stdin` **(Standard Input - FD 0):** Input data stream (default: keyboard).
2. `stdout` **(Standard Output - FD 1):** Normal output stream (default: terminal screen).
3. `stderr` **(Standard Error - FD 2):** Diagnostic and error logging stream (default: terminal screen).

23. Redirection Operators (`>`, `>>`, `2>`, `&>`)

- `>` **(Overwrite Redirect):** Redirects `stdout` to a file, completely erasing prior content: `echo "Job Started" > log.txt`
- `>>` **(Append Redirect):** Redirects `stdout` to a file, preserving previous lines and appending to the bottom: `echo "Epoch 1 Complete" >> log.txt`
- `2>` **(Error Redirect):** Redirects only `stderr` (errors) to a log file: `./simulation 2> error.log`
- `&>` **or** `2>&1` **(Merged Redirect):** Combines both `stdout` and `stderr` into a single file: `./simulation > combined.log 2>&1`

24. Here-Document (Heredoc `<< 'EOF'`)

- **Definition:** A multi-line string literal redirection syntax used in Bash scripts to generate configuration files or nested scripts on the fly.
- **Quoting Note:** Using `<< 'EOF'` (with quotes) prevents variable expansion inside the block; using `<< EOF` (without quotes) allows variables like `$USER` to be evaluated before writing.

25. Defensive Bash Scripting (`set -euo pipefail`)

The gold standard for enterprise production scripts:

- `set -e` **:** Exit immediately if any command exits with a non-zero (failure) status.
- `set -u` **:** Treat unset variables as an error and exit immediately (prevents accidental disasters like `rm -rf $DIR/` when `$DIR` is undefined!).
- `set -o pipefail` **:** If any command in a pipeline fails (e.g., `cmd1 | cmd2`), the exit status of the entire pipeline is the failure code, not the status of the last command.

26. Command Substitution (`$(command)`)

- **Mechanism:** Executes `command` in a subshell, captures its standard output, and replaces the expression with the captured text string: `STAMP=$(date +%Y%m%d_%H%M%S)`

Module 5: Secure Shell (SSH) & Cluster Networking

27. SSH (Secure Shell) & Port 22

- **Definition:** A cryptographic network protocol for secure data communication, remote command-line login, and remote command execution between two networked computers.
- **Default Port:** TCP Port 22.
- **Core Benefit:** End-to-end encryption. Replaced insecure legacy protocols like Telnet and RSH, which transmitted passwords in cleartext over the wire.

28. Asymmetric Cryptography (Public Key vs Private Key)

SSH key authentication uses asymmetric key-pair mathematics:

- **Private Key (`id_ed25519`):** Stored strictly on your local machine. Must be kept secret. Permissions **must be** `600` (`-rw-------`). If world-readable, SSH refuses to connect!
- **Public Key (`id_ed25519.pub`):** Can be freely shared with the world. Placed on the remote server inside `~/.ssh/authorized_keys`.
- **How Authentication Works:** The server encrypts a random challenge using your Public Key; only the holder of the matching Private Key can decrypt and answer the challenge. Your password is never sent over the network!

29. Ed25519 vs RSA Keys

- **RSA (Rivest–Shamir–Adleman):** The legacy standard. Requires 4096-bit keys for modern security.
- **Ed25519:** Modern elliptic curve cryptography (Curve25519). Faster, mathematically cleaner, immune to side-channel timing attacks, and uses a tiny 256-bit key string. **Recommended default for all new HPC clusters.**
- **Generation Command:** `ssh-keygen -t ed25519 -C "shivam@gla_hpc"`

30. `~/.ssh/authorized_keys` & `known_hosts`

- `authorized_keys` **(On Remote Server):** The list of public keys authorized to log in as this user account.
- `known_hosts` **(On Local Client):** A database of cryptographic fingerprints of remote servers you have previously connected to. Prevents Man-in-the-Middle (MITM) attacks where an attacker impersonates the server IP.

31. Secure Copy (`scp`)

- **Definition:** Securely transfers files and directories across networked machines over an encrypted SSH connection.
- **Syntax:** `scp [source] [destination]`
  - Upload: `scp local_script.sh user@cluster_ip:~/`
  - Download: `scp user@cluster_ip:~/results.csv ./`

Module 6: Intel oneAPI Toolkit (Compilers & SYCL Programming)

32. Intel oneAPI Unified Programming Model

- **Definition:** An open, cross-architecture programming initiative created by Intel, based on open Khronos standards (SYCL), designed to eliminate proprietary vendor lock-in (such as NVIDIA CUDA).
- **Core Vision:** "Write once, run anywhere on heterogeneous silicon" — a single C++ source code can be compiled to target Intel CPUs, Intel Arc/Data Center GPUs, NVIDIA GPUs (via Codeplay plugins), and AMD GPUs.

33. The 2026 Unified Toolkit Reality

- **Historical Context:** Prior to 2026, Intel distributed two massive separate suites:
  1. *oneAPI Base Toolkit* (Compilers, Math Libraries, DPC++).
  2. *oneAPI HPC Toolkit* (Intel MPI, Intel Fortran, Cluster Checker).
- **2026.0+ Unified Standard:** Intel merged these suites into a single **"Intel oneAPI Toolkit"**. All compilers (`icpx` , `ifx`), communication libraries (Intel MPI), and performance profilers are unified under a single `/opt/intel/oneapi` tree.

34. `icpx` (Intel oneAPI DPC++/C++ Compiler)

- **Definition:** Intel's modern LLVM-based C++ compiler with native support for the SYCL heterogeneous computing standard.
- **Replaced:** The legacy classic proprietary compiler `icpc` .
- **Primary Flags:**
  - `-fsycl` : Enables SYCL runtime integration and generates device-specific SPIR-V kernels.
  - `-O2` / `-O3` : Production level code optimization, vectorization, and loop unrolling.
  - `-o [name]` : Specifies output binary name.

35. Khronos SYCL Standard

- **Definition:** A royalty-free, cross-platform high-level C++ programming model that enables code for heterogeneous devices (CPUs, GPUs, DSPs, FPGAs) to be written in modern, standard ISO C++.
- **Key Characteristics:** Single-source (host code and device kernels live in the exact same `.cpp` file), template-driven, and asynchronous task-graph based.

36. SYCL Core Primitives (`queue` , `buffer` , `accessor` , `handler`)

- `sycl::queue` **:** The abstraction through which work (kernels) is submitted to an execution device.
- **Device Selectors:**
  - `sycl::cpu_selector_v` : Directs the runtime to execute the kernel on CPU cores.

    - `sycl::gpu_selector_v`: Directs the runtime to execute the kernel on a GPU.
    - `sycl::default_selector_v`: Automatically picks the highest-performance available device.
- **`sycl::buffer`:** Manages 1D, 2D, or 3D contiguous memory across both host (system RAM) and device (GPU VRAM) automatically handling PCIe/CXL data migration.
- **`sycl::accessor`:** Defines *how* a kernel accesses buffer data (`read`, `write`, `read_write`). Accessors construct the Directed Acyclic Graph (DAG) dependencies for the SYCL runtime scheduler.
- **`parallel_for`:** Dispatches a computational lambda across an N-dimensional iteration space across all physical hardware vector units.
- **`q.wait_and_throw()`:** Synchronizes the host thread, waiting for all asynchronous device operations to complete, and catches hardware execution exceptions.

37. `sycl-ls`

- **Definition:** The official oneAPI command-line utility that queries and lists every compute target visible to the SYCL runtime on the active machine.
- **Sample Output:**

```
[opencl:cpu:0] Intel(R) OpenCL, 11th Gen Intel(R) Core(TM) i3-1115G4
[ext_oneapi_level_zero:gpu:0] Intel(R) Level-Zero, Intel(R) UHD Graphics
```

38. `setvars.sh`

- **Definition:** The official Intel environment initialization shell script located at `/opt/intel/oneapi/setvars.sh`.
- **Mechanism:** Inspects installed oneAPI components and modifies the active shell environment: prepends compiler binaries to `$PATH`, shared objects to `$LD_LIBRARY_PATH`, and cmake configurations to `$CMAKE_PREFIX_PATH`.
- **Execution Syntax:** `source /opt/intel/oneapi/setvars.sh` (Must use `source`, not `./`, so changes persist in the active shell!).

Module 7: Intel HPC Toolkit (MPI, Fortran & Profiling)

39. MPI (Message Passing Interface)

- **Definition:** The de facto international industry standard for distributed-memory parallel computing across independent networked compute nodes.
- **Model:** Shared Nothing / Distributed Memory. Each process has its own isolated memory space. Communication occurs via explicit message passing across the interconnect network.

40. Intel MPI Library & `mpirun` / `mpicxx`

- **`mpicxx`:** The compiler wrapper that automatically links Intel MPI headers (`mpi.h`) and shared libraries with the underlying C++ compiler (`icpx` or `g++`).
- **`mpirun` (or `mpiexec`):** The process launcher that spawns N instances of the compiled binary across local CPU cores or remote compute nodes.
- **Basic Launch Syntax:** `mpirun -n 4 ./mpi_program` (Launches 4 parallel processes).

41. Key MPI Primitives (`Rank`, `Size`, `Communicator`)

- **Communicator (`MPI_COMM_WORLD`):** The default communication group containing all processes launched by `mpirun`.

- **Rank:** The unique integer ID assigned to each process in the communicator (ranges from `0` to `Size - 1`).
  - Rank 0 is conventionally designated as the "Master" or "Root" process responsible for I/O and coordination.
- **Size:** The total number of processes participating in the communicator.

42. `ifx` (Intel Fortran Compiler)

- **Definition:** Intel's modern LLVM-based Fortran compiler supporting Fortran 77, 90, 95, 2003, 2008, and 2018 standards with OpenMP offloading.
- **Replaced:** The legacy classic compiler `ifort`.
- **Classroom Context:** Essential because >60% of legacy scientific HPC simulations (weather forecasting, CFD, molecular dynamics) are written in Fortran.

43. Intel VTune Profiler

- **Definition:** An advanced hardware performance profiling tool for CPUs, GPUs, and cluster applications.
- **Capabilities:**
  - Hardware event sampling via Performance Monitoring Units (PMU).
  - Identifies **Hotspots** (functions consuming the most CPU cycles).
  - Detects **Memory Bandwidth Saturation** and cache misses (L1/L2/L3).
  - Identifies lock contention, thread imbalance, and False Sharing.

🎯 Quick-Reference Cheat Sheet for Common Student Questions

Student Question	Short Technical Answer (Trainer Script)
"Sir, why not just use Python instead of C++ for HPC?"	"Python is an interpreted, dynamically-typed language with a Global Interpreter Lock (GIL). It is 50x to 100x slower for raw computation. Libraries like PyTorch and NumPy are only fast because their underlying engines are written in C++ and compiled with oneAPI!"
"Sir, what is the difference between CUDA and oneAPI?"	"CUDA is proprietary to NVIDIA; code written in CUDA cannot run on Intel or AMD chips. oneAPI is an open industry standard based on SYCL; the same code can run across Intel, NVIDIA, and AMD hardware."
"Sir, why do we use `chmod 755` instead of `chmod 777`?"	"`777` grants write permissions to *everyone* on the network, which is a massive security hazard. `755` ensures only you (the owner) can modify the script, while others can only read and execute it."
"Sir, what is the difference between a core and a thread?"	"A core is an actual physical silicon circuit with its own registers and ALUs. A thread (Hyper-Thread) is a software execution stream scheduled by the OS. One physical core can juggle two threads using SMT."
"Sir, does `>` append to a file?"	"No! `>` wipes the file completely and overwrites it. To append safely without deleting existing text, always use `>>`!"

Document Status: Complete & Verified | Ready for Classroom Delivery | GLA University HPC Masterclass EOF